

CS5242 Project Report

Group 51

[redacted] ([redacted])
[redacted] ([redacted])

April 16, 2018

1 Task Description

We are given a training dataset consisting of 110,000+ binary sequences, extracted from the Portable Executable (PE) header of executable files. The size of these files are not fixed, e.g. some files have 512 bytes and others have 1024 bytes or more. Each file has a binary label, 0 for benign and 1 for malware. Our task is to train a deep learning model for the binary classification problem of malware detection.

2 Highlights of Solution

We consider using deep neural networks on the raw byte sequences in the PE headers. In particular, we consider using the Multi-Layer Perceptron and the Recurrent Neural Network (Long Short Term Memory) architectures.

Highlights of our implementation include making use of input sequence truncation, an attention mechanism, and ELUs, which we detail in Section 4.

3 Related Work

While it may be recommended to employ domain knowledge to machine learning problems in general, for the malware classification problem, the converse may be true due to the complicated nature of executable files. Domain knowledge here refers to knowledge used in data processing, feature construction and selection within specific problem domains.

For malware detection there exist two common approaches without domain knowledge, namely byte n-grams and strings. In this project, we make use of the raw byte sequence of the PE header without domain knowledge nor explicit feature construction, and use deep learning models to learn from these raw byte sequences. We follow closely the approach taken in Raff et. al [1]

4 Theory & Model Introduction

4.1 Input Sequence truncation & Padding

We truncate the sequence of bytes which are longer than a specific value, which we test as a hyperparameter, and pad with the value 0 all sequences which are shorter than this value. We experimented with different values and decided to test values of 512 and 1024 during model selection, mainly taking into account the mean length of sequences in our dataset.

4.2 Embedding Layer

We feed the entire truncated sequence of bytes through an embedding layer, the output of which is then fed into the fully connected or LSTM network. The embedding layer maps each byte value into a unique feature vector in the embedding space $\mathbb{R}^B$, where B is a number smaller than the dimension of the inputs.

As a heuristic, we find that taking B as the fourth root of the value of the dimension of the inputs tends to produce better results [3].

4.3 Fully Connected Neural Network

We implement the fully connected neural network as suggested in [1], with experimental tweaks detailed in Section 5.

4.4 Long Short Term Memory

We implement the LSTM model, which was first introduced in [2]. The main idea behind Recurrent Neural Networks is to make use of sequential information; the networks allow information from previous time steps in a sequence to persist. The LSTM variant of RNN tackles the vanishing gradient problem and remembers information for long periods of time, whereas vanilla RNNs are unable to connect information at a certain time step to previous time steps far away. This capability of long-term dependency could well be crucial to byte sequence classification.

An LSTM contains the cell state, C_i and outputs the hidden state h_i at each time step. It has three gates, to control the cell state and to decide what to output. Firstly, to decide what information to throw away at each step, we look at h_{t-1} and the current input, x_t , and we have the forget gate:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (1)$$

To decide what new information at each time step to store, we have the input gate and the update candidate values, $\tilde{C}_t$:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (2)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (3)$$

Then we update the cell state:

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad (4)$$

Finally, we decide what to output using the output gate:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (5)$$

$$h_t = o_t * \tanh(C_t) \quad (6)$$

4.5 Attention Mechanism

The attention mechanism is a useful technique in neural networks, particularly in RNNs. It allows a neural network the ability to focus or de-focus on subsets of the inputs, and in RNNs' case subsequences of the inputs. We describe our implementation of the attention mechanism for our LSTM model which we follow from [1].

Given h_i , the hidden activation of the last LSTM layer at step i , T steps of the LSTM, we compute

$$\tilde{\alpha}_i = v^T \tanh(W_0^T h_i + W_1^T \bar{h} + b), \text{ where } \bar{h} = \frac{1}{T} \sum_{i=1}^T h_i \quad (7)$$

Here $\bar{h}$ encodes information across the entire sequence; W_0 and W_1 considers the contribution of local and global hidden activations respectively. The vector v is then used to obtain the unscaled $\tilde{\alpha}_i$ at each time step, before applying the softmax function to obtain the scaled importance α_i . Finally, we get the output of the attention mechanism as:

$$\text{Attention output} = \sum_{i=1}^T \alpha_i h_i, \text{ where } \alpha_i = \frac{\exp \tilde{\alpha}_i}{\sum_{j=1}^T \tilde{\alpha}_j} \quad (8)$$

4.6 Dropout

We apply dropout for regularization with 0.45 probability on hidden layers (FC/LSTM) and 0.20 probability on the embedding space. We started with the values suggested in [1] and after tweaks arrive at these values.

4.7 Exponential Linear Unit

After each FC layer we make use of ELU activation function which can be expressed as

$$ELU(x) = \begin{cases} x, & \text{if } x \geq 0 \\ \alpha(e^x - 1), & \text{otherwise} \end{cases} \quad (9)$$

We observe faster convergence for ELU as compared to the RELU activation function, because the latter has a positive mean output, while the former has a mean activation close to 0 and allows faster learning.

5 Experiment Setting

Machines We initially tried NSCC and Amazon EC2 for model training. However, due to unpredictable turnaround times and the unavailability of GPU instances on Amazon EC2, we finally chose Google Cloud Platform’s ML Engine, where training was done on Nvidia Tesla K80 GPUs (Memory: 11.17 GiB, Clock Rate(GHz): 0.8235, compute capability: 3.7).

We also had access to a local Nvidia GeForce GTX 960M GPU (Memory: 2048 MB, Clock Rate(MHz): 1096, compute capability: 5.0). As a comparison, training on the cloud GPU was generally about twice as fast as on the local one.

Training/Validation Split From the training dataset, we held out 1000 training examples for validation (validation_split = 0.0088), and trained our models on the rest of the examples.

Hyperparameter tuning Hyperparameters of interest included architecture parameters (type of model, number of layers, layer sizes), the learning rate, and regularisation parameters. In the case of architecture changes, we followed a general approach of varying parameters and training the corresponding models for 5-10 epochs, then choosing the setting with the best result. For each chosen model, other parameters such as learning rate could be more easily tuned while training. The most significant experimental results on architecture tuning are shown below:

- **FC Network:** The number of layers was varied from 4-7, and number of nodes per layer was varied between 32 and 64. The following figures show the changes in metrics over 9 epochs for the different architectures (the first epoch was omitted to display differences in curves more clearly). To balance between performance and training time constraints, we chose to continue tuning and training the 4-layer and 6-layer models with 64 nodes.

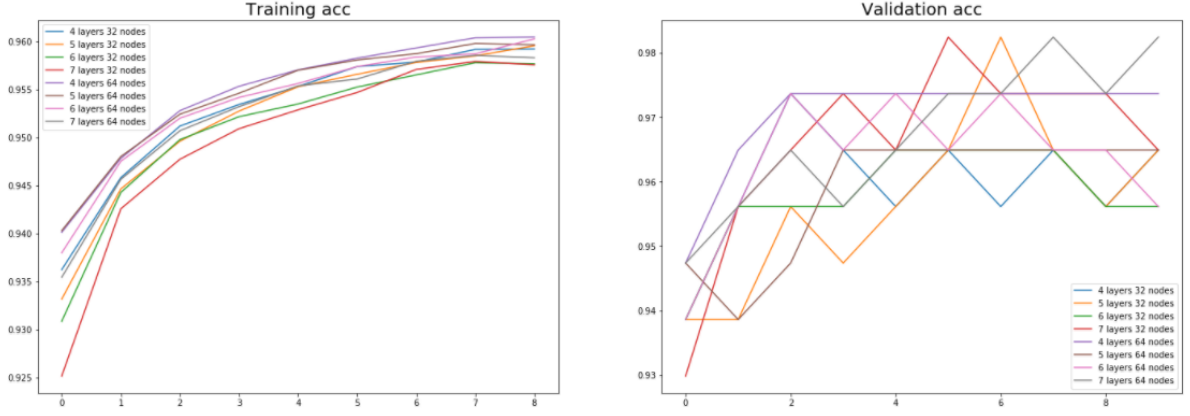


Figure 1: Accuracies over epochs for different FC architectures

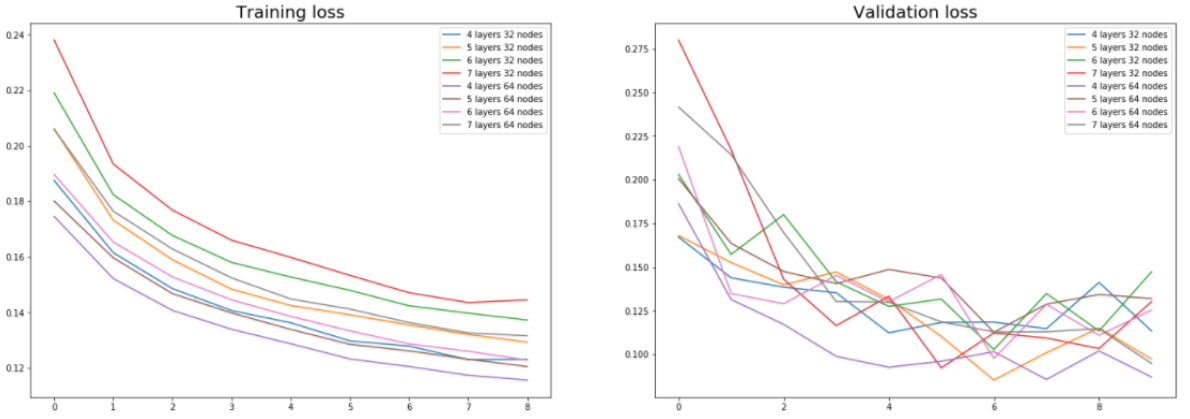


Figure 2: Losses over epochs for different FC architectures

- **LSTM:** The number of layers was kept constant at 3, while the number of nodes per layer was varied between 16 and 32, and the input truncation was varied between 512 and 1024 bytes. The following figures show the changes in metrics over 5 epochs for the different architectures. Notably, performance was poor for models using a sequence length of 1024, possibly due to vanishing or exploding gradient problems, or the high number of parameters. To balance between performance and training time constraints, we chose to continue tuning and training the best-performing model with 32 nodes per layer and input length of 512 bytes.

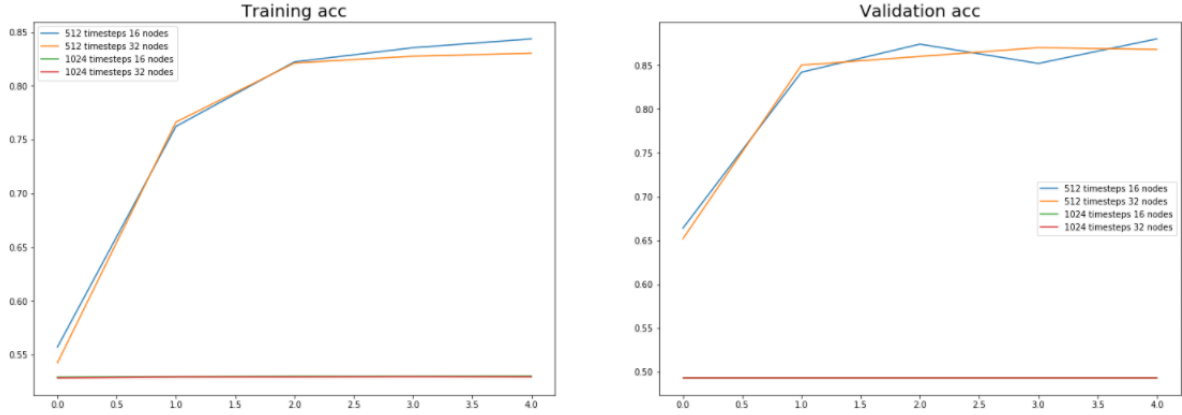


Figure 3: Accuracies over epochs for different LSTM architectures

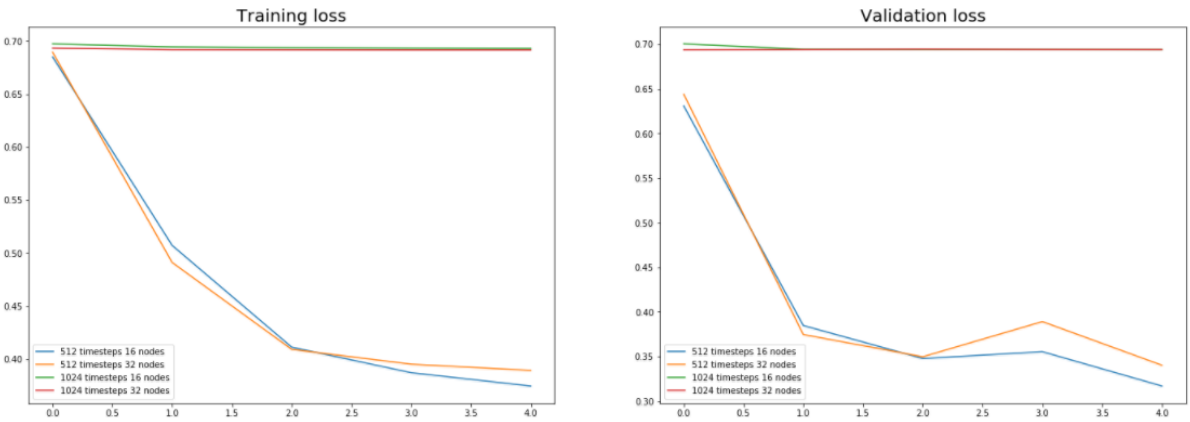


Figure 4: Losses over epochs for different LSTM architectures

The following sections contain further details on the settings chosen for the best-performing model, a 6-layer FC network.

6 Result Analysis

6.1 FC Network Results

Metrics After 300 epochs of training, the best final model submitted to Kaggle had a validation accuracy of 0.9737 (val loss: 0.1149). Further training beyond this point had negligible effects on the model. On the Kaggle test data, the model achieved an Area Under the Receiver Operating Characteristic curve (AUC) value of 0.98717.

Architecture The best-performing model was a 6-layer FC model derived from the suggestions in [1]. This architecture consists of an initial embedding layer (output dimension = 4), followed by 6 fully-connected dense layers with 32 nodes each.

The main improvement to the original architecture was reducing the embedding dimensions, based on a guideline [3] to set the embedding vector dimension to the 4th root of the number of categories: This change improved training speed, and increasing the embedding dimensions further did not increase the validation accuracy or test AUC reported by Kaggle.

Batch size Training was faster with a larger batch size, therefore $N = 1024$ was used in the final model. Sizes beyond this risked causing out-of-memory errors.

Learning rate and Optimiser Different initial values and decay methods were tried. The final settings were $lr = 1e-4$, no decay, and setting the scheduler function to multiply the learning rate by 0.95 every epoch. This gave good results in the initial 200 epochs of training. For subsequent epochs, the learning rate was manually tuned, by observing whether the values caused the losses were decreasing. For the optimiser, Adam was used with default settings for β_1 (0.9) and β_2 (0.999).

Regularisation L2 regularisation was used in each layer, with different values of λ for different types of layers (Embedding: $1e-4$, Dense: $2e-5$). Dropout was also used, at a value of 0.2 for the Embedding layer and 0.45 for all other dense layers. The values of λ were chosen such that in later stages of training, the regularisation loss was often within 1-2 orders of magnitude of the training loss, so that neither would dominate the loss term as the model converged.

Other settings Batch Normalisation was also applied in each layer.

6.2 LSTM

Metrics The LSTM model was trained for 30 epochs, reaching validation accuracy of 0.9712 (val loss: 0.1208). On the Kaggle test data, the model achieved an Area Under the Receiver Operating Characteristic curve (AUC) value of 0.98608.

Architecture The LSTM model followed the architecture suggested in [1], with 3 LSTM layers followed by an attention mechanism and a final dense layer with sigmoid activation. We used 32 nodes in each LSTM layer and 512 time-steps each.

Batch size Training was faster with a larger batch size, therefore $N = 1024$ was used in the final model. Sizes beyond this risked causing out-of-memory errors.

Learning rate and Optimiser Different initial values and decay methods were tried. The final settings were $lr = 1e-3$, no decay, and setting the scheduler function to multiply the learning rate by 0.95 every epoch. This gave good results in the initial 200 epochs of training. For subsequent epochs, the learning rate was manually tuned, by observing whether the values caused the losses were decreasing. For the optimiser, Adam was used with default settings for β_1 (0.9) and β_2 (0.999), as well as gradient clipping of 1.0.

Regularisation The embedding layer used a dropout of 0.2, and all other layers had a dropout of 0.5. In addition, LSTM layers had L2 regularisation with λ of $1e-5$, while other layers used L2 regularisation with λ of $1e-4$.

6.3 FC vs LSTM

We find that the FC neural network outperforms LSTM, and this could be due to the difficulty of training an LSTM well. Possible reasons include the long length of the sequence (1024), which could cause a problem with vanishing or exploding gradients.

6.4 Attention Mechanism

In our experiments, we find that multi-layer attention networks did not work as well as the case when a single layer attention is used. This could be due to the increased number of parameters.

7 Possible Improvements

Possible improvements include using truncated BPTT for faster training and lower risk of vanishing or exploding gradients, and exploring the use of attention in an FC network. Also, many other architectures could be explored, such as networks with residual blocks, convolutional networks, and bi-directional recurrent networks.

References

- [1] E. Raff, J. Sylvester, and C. Nicholas, Learning the PE Header, Malware Detection with Minimal Domain Knowledge, ArXiv e-prints, Sep. 2017.
- [2] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735-1780, 1997.
- [3] Introducing TensorFlow Feature Columns. Google Developers Blog, Nov 2017. Retrieved from <https://developers.googleblog.com/2017/11/introducing-tensorflow-feature-columns.html>.